# Network-on-Chip Built-In Self Test for enhancing availability of Multi-Processor System-on-Chip

*A Project Report*

*submitted by*

**Shashank V (15EC147)**

*under the guidance of*

**Prof. S.K Nandy**

**Convener,**

**Computer-Aided Design Lab,**

**Indian Institute of Science, Bangalore**

*in partial fulfilment of the requirements*

*for the award of the degree of*

**BACHELOR OF TECHNOLOGY**




**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**

**NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA**

**SURATHKAL, MANGALORE - 575025**

**November 26, 2018**

# ABSTRACT

Many-core systems are very popular in the current scenario of increasing requirement for computation power by data intensive applications. Embedded processors with 64, 256 or even 1024 cores are becoming a necessity in the face of large amounts of data. However, with the increase in number of cores, there exists a problem with the universally used bus system for transferring addresses and data from one core to another. It has been demonstrated that performance of a bus degrades as the number of processing cores dependent on it moves beyond 4. Due to this, a different architecture is needed that suites the demands of a large number of cores. Network-on-Chip (NoC) is one such architectural alternative which provides the high-bandwidth communication substrate required for the operation of such an MP-SoC. An NoC is an interconnection of routers connected to each other in a specific pattern. The pattern can be either regular or irregular, depending on the level of customization required. These routers have 5 input/output ports, one for each direction (N, S, E, W), and an ingress/egress port. Processing cores are connected to each router through the ingress/egress port.

These systems are often used in safety-critical applications. In such cases, availability of the NoC must be known before launching applications. Since there are many different components in the NoC, such as the communication links, the router, and the processing core, it is useful to test each component in a recursive manner, i.e using already tested resources for further testing of other components. The communication links form the backbone of the NoC, and are used for all transfer of data between the processing cores. Therefore, the testing procedure is first carried out for the links. Once this data is collected, it can be used to test the other components in a systematic manner. In this project, the type of faults considered are open and stuck-at faults. A distributed algorithm is employed to collect information about status of links. Information from all the routers is collected at a central control unit, which does availability analysis in software using this information. The result of this process is determination of available regions of the NoC.

## DECLARATION

I, Shashank V, a student of seventh semester Bachelor of Technology in Electronics and Communication Engineering at National Institute of Technology Karnataka, Surathkal, hereby declare that the project titled 'Network-on-Chip Built-in-Self-Test for enhancing availability of Multi-Processor System-on-Chip' and the work presented in it are my own and submitted as partial fulfillment for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering during the academic year 2018-2019. The material contained in this report has not been submitted to any University or Institution for the award of any degree.

Place : Surathkal

Date : Nov 26, 2018

Shashank V

## ACKNOWLEDGEMENTS

I am indebted to my guide, Prof. S.K Nandy, for providing me the opportunity to work on my undergraduate major project at IISc. His constant encouragement and technical insights were invaluable throughout the duration of my project. I would like to thank Dr. Ranjani Narayan, Chief Technology Officer, Morphing Machines Pvt. Ltd. for her support and continued guidance which were crucial throughout the project. I would also like to thank Dr. Chandan Haldar, Chief Executive Officer, Morphing Machines Pvt. Ltd., for his guidance and constructive criticisms, which helped me in improving the project.

I would also like to thank Madhava Krishna, Swarali Patil, Ritika Singh, Anoop M.N and Abhishek Tyagi for their help and stimulating discussions.

I would like to thank the Department of Electronics and Communication Engineering and NITK Surathkal for giving me the opportunity to get a taste of research during my undergraduate studies.

I would like to express my sincere thanks to Dr. Arulalan M.R and Dr. Deepu Vijayasenan, for their help, feedback and suggestions. I would also like to thank the faculty of the Department of ECE for giving me a strong foundation to undertake this project.

Finally, I would like to thank my parents and friends for their constant support and motivation.

# TABLE OF CONTENTS

**ABSTRACT** **i**

**1 Introduction** **1**

1.1 Problem definition . . . . . 1

1.2 Previous work . . . . . 2

1.3 Motivation . . . . . 3

1.4 Overview . . . . . 4

**2 NoC Basics** **6**

2.1 Architecture . . . . . 6

2.2 Routing Algorithm . . . . . 7

**3 Analysis of availability** **9**

3.1 Built-In Self Test - Ping Pong process . . . . . 9

3.2 Consolidation . . . . . 16

3.3 Decision Making . . . . . 18

**4 Results and Examples** **22**

4.1 Execution time for ping pong process . . . . . 22

4.2 Execution time for consolidation process . . . . . 23

4.3 Implementation and hardware overheads . . . . . 25

4.4 Example 1 - Faulty vertical inlink in gateway column . . . . . 26

4.4.1 Case 1 - Broken south inlink of router G3 . . . . . 26

4.4.2 Case 2 - Stuck-at fault in south inlink of router G3 . . . . . 27
4.4.3 Case 3 - Broken south inlink of router G1 . . . . . 29
4.4.4 Case 4 - Stuck-at fault in south inlink of G1 . . . . . 29
4.5 Example 2 - Faulty horizontal inlink in first row . . . . . 30
4.5.1 Case 1 - Broken east inlink of router 3 . . . . . 30
4.5.2 Case 2 - Stuck-at fault in east inlink of router 3 . . . . . 30
4.5.3 Case 3 - Broken east inlink of router 5 . . . . . 30
4.5.4 Case 4 - Stuck-at fault in east inlink of router 5 . . . . . 31
4.6 Example 3 - Faulty west inlink of router G1 . . . . . 32
4.6.1 Case 1 - Broken west inlink of router G1 . . . . . 32
4.6.2 Case 2 - Stuck-at fault in west inlink of router G1 . . . . . 33
4.7 Example 4 - Faulty east outlink of router 11 . . . . . 34
4.7.1 Case 1 - Broken link . . . . . 34
4.7.2 Case 2 - Stuck-at fault . . . . . 34
4.8 Example 5 - Faulty north outlink of router 15 . . . . . 34
4.8.1 Case 1 - Broken link . . . . . 35
4.8.2 Case 2 - Stuck-at fault . . . . . 35
4.9 Example 6 - Faulty east outlinks of routers 11 and 17 . . . . . 36
4.9.1 Case 1 - Broken link . . . . . 36
4.9.2 Case 2 - Stuck-at fault . . . . . 36
4.10 Example 7 - Faulty east and west inlinks of router G1 . . . . . 37

**5 Conclusions** **38**

5.1 Analysis . . . . . 38
5.2 Future Work . . . . . 38

# LIST OF FIGURES

1.1 An example fabric with two applications launched on it . . . . . 2

2.1 Internal architecture of router . . . . . 7

3.1 Router links and status register . . . . . 10

3.2 Example of a ping-pong process with one faulty link . . . . . 14

3.3 Example of a pass-through router . . . . . 19

4.1 Number of cycles required for ping pong process with fabric scaling . . 22

4.2 Number of cycles required for consolidation process with fabric scaling . 24

4.3 Virtual divisions of the fabric due to X-Y routing . . . . . 26

4.4 Availability analysis of Example 1, Case 1 . . . . . 28

4.5 Availability analysis of Example 1, Case 3 . . . . . 29

4.6 Availability analysis of Example 2, Case 1 . . . . . 31

4.7 Availability analysis of Example 2, Case 3 . . . . . 32

4.8 Availability analysis of Example 3, Case 1 . . . . . 33

4.9 Availability analysis of Example 4, Case 1 . . . . . 35

# CHAPTER 1
# Introduction

## 1.1 Problem definition

Network-on-Chip is a popular choice of communication infrastructure for Multi-Processor System-on-Chip (MP-SoC). Large number of processing cores can be included in the system by choosing a suitable interconnection of routers, called execution fabric. The limitation of number of cores on a single bus does not arise here. Further, due to its very structure, the NoC offers a high degree of parallelism, so that many different applications can be operated on at the same time. Many of such applications can be safety-critical ones, which are required to go to completion. This necessitates a fault-identification and fault-tolerant mechanism, thereby ensuring that applications complete and produce the right results.

Figure 1.1 shows an NoC with a 6*6 execution fabric and one column of routers called gateway routers. The NoC is implemented as a bidirectional toroidal mesh topology. It is bidirectional because communication between routers is full duplex, and the toroidal topology ensures that there is continuity between routers without abrupt edges. The processing cores, also called as compute resources (CR) are attached to the ingress/egress ports of each router as shown. There is a control unit shown in yellow, which is the interface between the execution fabric and the outside world (host). It is the control unit which is responsible for allocating resources as the host requires, and for collecting the final information at the end of application execution and communicating it back to the host. To maintain uniformity, none of the other gateway routers are connected with processing cores.

The word kernel used in the figure is synonymous with application. For applications to reach completion, it is essential that all components of the NoC, such as routers, communication links, compute resources, are fault-free. Thus, a test mechanism is required to verify the working of these components and convey the results to the control unit, so that it launches applications only on fault-free regions of the execution fabric. This testing can be done in a recursive manner, by using components that are already tested to verify the working of other components. In the case of an NoC, the communication

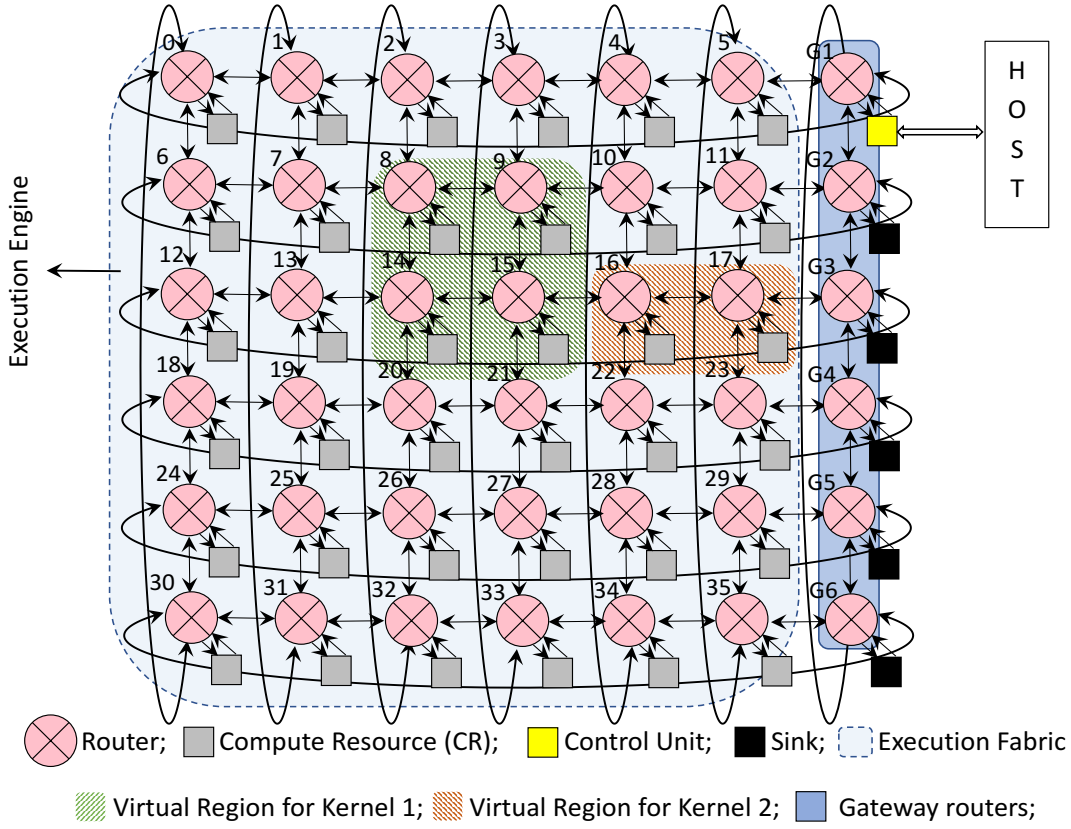



Figure 1.1: An example fabric with two applications launched on it

links between routers are critical to its working. Therefore, it follows that link testing should be conducted as the first stage of NoC testing. Once tested, the fault-free links can be used to test other components like the compute resources. For the purposes of this project, the internal components of the router, such as the buffers and the routing algorithm are assumed to be fault-free and are used in the testing of communication links.

## 1.2 Previous work

Over the past years, many authors have presented different test strategies for Network-on-chip. However, NoC as a complex subsystem of a MPP, has various components of the communication infrastructure that need to be tested individually.

Grecu et al in (1) propose a BIST for Network-on-Chip Interconnect Infrastructures, that focuses on testing the inter-switch links of the chip by progressive reuse of NoC infrastructure. The basic building blocks involved include a test data generator which launches the test vectors and a test error detector which samples and compares the received data. Grecu et al in another work (2) propose a similar method that also exploits

the inherent parallelism of the data transport mechanism to reduce the overall test time, but addresses testing of both switches and inter-switch links. Raik et al in (3) talk about testing configurations for functional switch faults but like most of the previous work on testing, the focus is on testing the faults inside the switches. The testing method used in (4) by Mohammad Hosseinabady et al broadcasts the test vectors throughout the network. The algorithm proposed follows the scan-based methodology for testing the switches. In fact, majority of the previous work in this field involves scan-based techniques like the ones mentioned in (5), (6).

Peterson et al in (8) present a BIST strategy for testing the NoC interconnect network having mesh topology. This paper has outlined a technique that identifies the healthy regions of the fabric, which is similar to our objective. In their method, the BIST modules in each Network Interface perform the testing of the datapath and control logic. It is left to the operating system to analyze the information, identify portions of the fabric which are fault-free, and isolate the same.

## 1.3 Motivation

The objective of the project is to propose an extendable BIST for NoC for very large MPPs, so as to determine their availability for safety critical applications. However, this does not seem to be the focus of most of the above-mentioned endeavours.

One exception is the method presented by Peterson et al. The objective of the paper is to identify fault-free regions in the NoC, which is similar to the objective of this project. There is a major difference however, in the communication of routers with the outside world. All the routers are connected to the outside world via IO ports in the above paper, unlike here where there is a single point of communication with the outside world (control unit). This implies that in this project, links of the NoC are critical to communication between routers and control unit, whereas in the paper, the NoC is only used for on-chip communication. Further, the strategy for identifying healthy regions of the NoC has not been clearly outlined.

Due to the objective discussed in this project not being captured entirely in previous work done in this field, there is a need for an algorithm which targets determining the availability of a system.

## 1.4 Overview

The NoC used is a bidirectional toroidal mesh interconnection of routers, following deterministic X-Y routing for communication between routers. The X-Y routing is explained in a subsequent section. Each router has five input/output ports, one for each direction (N,S,E,W) and one for the connection to a CR. An arbitration logic exists to connect different input and output ports to enable the flow of data. Feedback connection is forbidden, i.e input and output ports in one direction cannot be connected together. A routing function is present in each of the routers and is the component responsible for determining the required output port for the data.

Communication takes place in the form of packets. A packet is a collection of bits containing information that a CR can understand and use to take appropriate action. Each packet has a header which contains the destination information in terms of delX and delY (relative addressing). Since packets can be quite long in size, they are further broken down into multiple flits, which are the data that actually flow through the physical links. For the conversion of packets into flits and vice versa, there exist "packetizer" and "flitizer" modules at each ingress/egress port.

Applications are launched onto the NoC upon a request from the host/user. The control unit identifies the number of resources needed for successful execution of an application, allocates and book-keeps resources assigned to it. While allocating resources to an application, the control unit gathers together physically proximal CRs and creates a virtual region that is entirely devoted to the application (for the period of execution); refer to Figure 1.1 for example of virtual regions indicated by hatched rectangles.

The testing can be classified as Built-In Self Test (BIST), since the control unit is part of the system and the user (host) does not determine how and in what order the testing is supposed to happen. The BIST is implemented as a micro-architectural feature inside each router, as explained in subsequent sections. This BIST can be performed either at cold start (when the system is powered on), or at any other point of time, subject to the completion of running applications. A reset command from the user will trigger the BIST and the control unit will have access to information regarding healthy parts of the fabric at the end of the BIST phase.

The first task is to identify faulty communication links. There are many different fault

models available, catering to faults like stuck-at faults, short faults etc. As a first step, it is assumed that a fault in a link implies that the physical link is broken. This would result in data not being transferred from one router to another. The testing procedure is then extended to accommodate stuck-at faults by relying on different test patterns to recognize incorrect data and hence identify the fault.

In order to identify a faulty link, a distributed algorithm is proposed in which test packets are sent from each router in a particular direction and a response packet is sent by the router which receives the test packet. The same procedure is applied to the remaining three directions. Status registers for the incoming link and the outgoing link are present in the router, and are updated during this process. At the end, each router has information on the state of all the links directly connected to it. This information can include some false negatives (link incorrectly marked as faulty), but it does not affect the final decision, as will be explained later.

Being a distributed network, an optimal way of consolidating this information at the control unit would be to have the routers send their information to the control unit at the end of a specified timeout. The control unit either receives or does not receive the information from a router, depending on the pattern of faults, and updates this information in the memory.

The decision making stage involves identifying the routers unreachable due to faults, and identifying a healthy portion of the fabric that can be used to run applications. Therefore, fault-tolerance is exercised in this method and the system is still able to guarantee completion of applications and correctness of results.

# CHAPTER 2
# NoC Basics

## 2.1 Architecture

Consider an m(rows)*n(cols) NoC, which is a distributed system with interconnection of routers. The word toroidal connection signifies the connection between the routers in the first and last column, as well as the routers in the first and last row. The presence of the toroidal connection offers a significant advantage in terms of time taken to reach a router at the opposite corner of control unit. Without the toroidal connection, a packet from the control unit to the router at the opposite corner would have to traverse a distance equal to (m + n) edges and therefore takes (m + n) cycles to reach the router (since 1 edge is traversed in 1 cycle). With the toroidal connection however, the packet needs to traverse only 2 edges and takes 2 cycles to reach the router.

The toroidal connection, however, has a potential to cause deadlocks. Let us consider an example where all routers in a row are sending packets in their west direction. In this case, every router is waiting for the next router to release the physical resource (link), thereby creating a dependency cycle. This situation can be resolved by having two virtual channels (VCs) for one physical channel. In such a case, the router in the first column (called dateline router) in the example above would send the packet to a different virtual channel, thereby freeing up the resources for the other packets to move through the row. This is done by changing the VC each time a flit moves out of the west port of the dateline router. Similarly, there are dateline routers in the first row, which would change the VC each time a flit moves out of the north port.

Figure 2.1 shows the internal architecture of a router. There are five input/output ports which are connected through a crossbar switch. This switch consists of five arbiters, one for each output port. Each arbiter takes in four input port wires (except for the input port in the same direction) and routes the appropriate input port to the output port, using the routing information present in the header of a flit. In this figure, the VCs are not shown to avoid cluttering. The two VCs are present inside the input port and are multiplexed to produce a single output as shown.

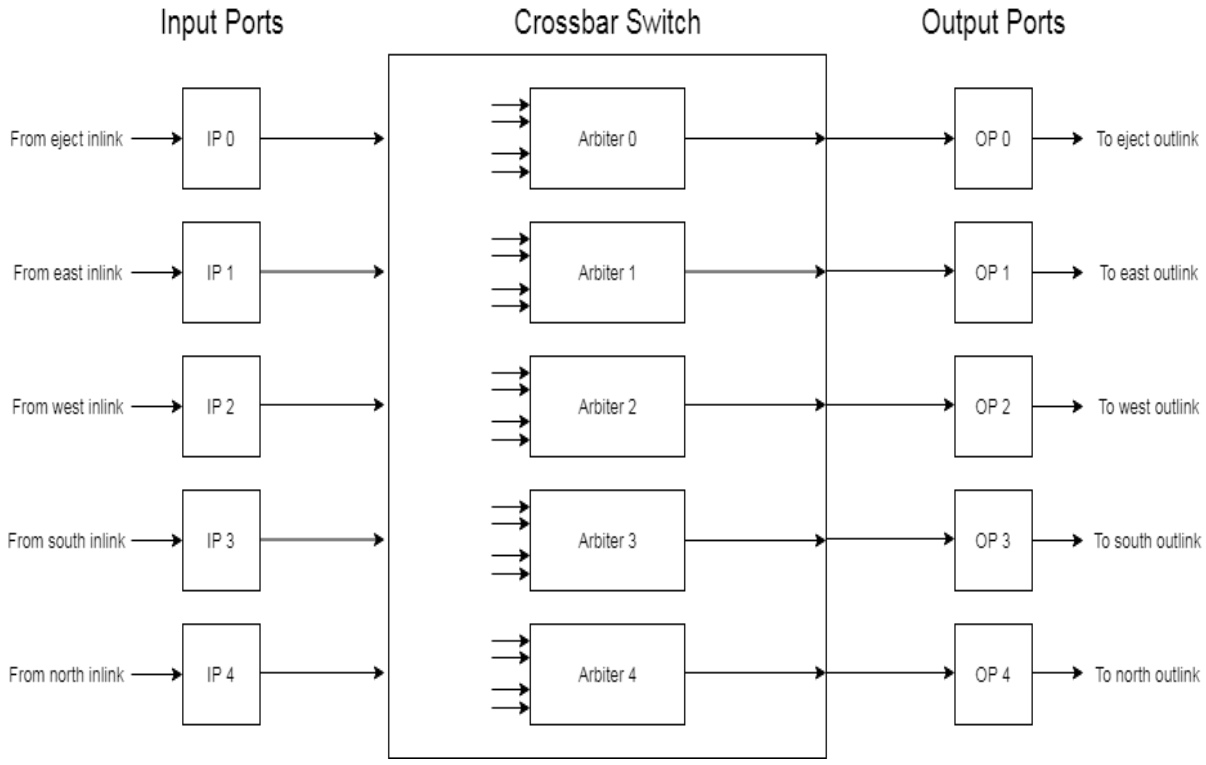



Figure 2.1: Internal architecture of router

## 2.2 Routing Algorithm

The routing algorithm used is deterministic X-Y routing. There are two reasons for choosing this type of routing. First, it results in a much simpler hardware compared to adaptive routing. Second, it ensures deadlock-free routing of packets. The routing algorithm is explained as follows. The relative distance between the source and destination is calculated and stored as $\Delta$X and $\Delta$Y. The routing follows the principle of minimizing $\Delta$X to 0 and then minimizing $\Delta$Y to 0. In the horizontal direction, a negative $\Delta$X implies that the flit has to travel in the west direction, and $\Delta$X has to be *incremented* at each intermediate router. A positive $\Delta$X implies hat the flit has to travel in the east direction, and $\Delta$X has to be *decremented* at each intermediate router. Similarly, in the vertical direction, a negative $\Delta$Y implies that the flit has to travel in the north direction, and a positive $\Delta$Y implies that the flit has to travel in the south direction. Once both $\Delta$X and $\Delta$Y are 0, the flit is routed to the egress port towards the CR.

The routing block inside input ports implement the above routing algorithm. When a flit containing the header information enters an input port, the $\Delta$X and $\Delta$Y are noted and the required output port is calculated. The flit then moves into the crossbar switch and is routed to the appropriate output port. At the output port, these delta values are

changed as required and the flit is sent to the next router.

The advantage of such a routing algorithm is that there is always a unique path from any source to any destination. This makes the routing deterministic in terms of the time taken for a flit to travel from one router to another. The paths from control unit to a router and vice versa are of particular importance. Due to the constraint of X-Y routing, the path from control unit to a router not on the same row or column as itself is not the same as the path from the router to control unit. As a result, there are two paths that govern the reachability of a particular router from the control unit.

# CHAPTER 3
# Analysis of availability

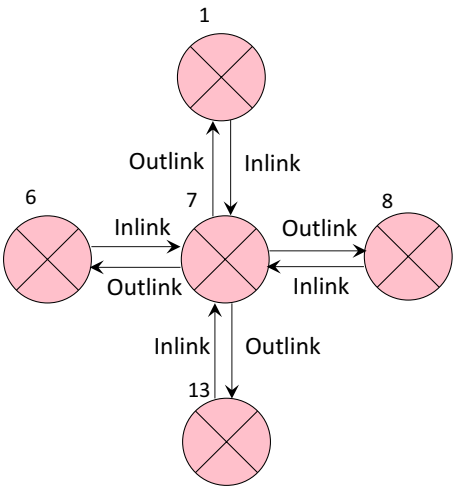
## 3.1 Built-In Self Test - Ping Pong process

With the hardware described above, a procedure is proposed to identify faulty links at each router, and take a decision at the control unit regarding the usable portion of the fabric. There are three stages in this procedure - Ping-Pong, Consolidation and Decision Making, explained below.

Figure 2.1 shows a router with outlinks and inlinks specified in all directions with respect to router 7. It is important to specify what is meant by inlink and outlink. For a particular router, the link which is transporting information from itself to the neighbouring router is called as outlink. Similarly, the link which is transporting information from a neighbouring router to the router in question is called as outlink. These terms are not absolute, i.e they are relative to the routers. For example, in the figure, the specifications of inlinks and outlinks are given with respect to router 7. From router 8's perspective, the link specified as router 7's outlink in the east direction is actually its inlink in the west direction. This difference will be apparent when tracing through the Ping-Pong process.

Both bidirectional links have a decoupled interface. This means that along with the information (bits), there are valid and ready signals. Bits and valid are signals in the same direction, whereas ready is a signal in the opposite direction. A high valid signal indicates that the information transferred over the bits signal is valid and vice versa. A high ready signal indicates that the neighbouring router is ready to receive information and vice versa.

Each router has a 10 bit status register, with 2 bits holding inlink and outlink status for each direction. All links are initialized as faulty (0), so that false positives do not occur. The state of the inlinks and outlinks in all directions can be updated at each router by sending packets and receiving a response. Testing of the ingress/egress links is slightly different from the testing of links in the other four directions. The reason and procedure for this is explained in a later section. For the other four directions, both broken and stuck-at links can be identified by a two-level check. The first level involves checking for a broken link. If required, the module then checks for stuck-at-1 and stuck-at-0 faults




Status of outlinks and inlinks of a router initialized as “unhealthy”

| Router 7 | N | S | E | W |
|---|---|---|---|---|
| Outlink | 0 | 0 | 0 | 0 |
| Inlink | 0 | 0 | 0 | 0 |

Status = 0 => Unhealthy
Status = 1 => Healthy

Figure 3.1: Router links and status register

and identifies if a link is healthy or not. Therefore, three ”test” packets are sent, and one ”response” packet is received. The reason for this non-uniformity is explained in the description of the procedure.

### Type and number of packets required for link status identification

In order to identify stuck-at faults, two test vectors are sufficient. These are the All 0’s vector and All 1’s vector. An All 0’s vector corrupted by a stuck-at-1 fault and an All 1’s vector corrupted by a stuck-at-0 fault can be identified by comparing the received flit with the expected flit. A fault would flip any of the bits in the vector and the comparison would fail, thereby identifying the link as faulty.

Some implementation details are required here to understand how information moves across links. As mentioned earlier, packets are broken down into flits and flits travel across the physical links. The information contained in a packet or a flit is called payload. There are four kinds of flits: HeadTail, Head, Body and Tail. HeadTail is used when the payload of the packet is small enough that it can fit in the payload of one flit. If this is not the case, a Head flit carries some payload bits of the packet, and Body (generated as required) and Tail flits carry the remaining bits. The only difference between a Body flit and Tail

flit is that the latter is used to indicate the end of a packet. Head and HeadTail flits contain the header of the packet, i.e the destination address and the VC Number.

The architecture of the router is such that it classifies a Body or Tail flit sent without a Head flit as a stray flit, and raises an assertion. The start of a packet should always be a Head or HeadTail flit. This is because these flits hold information about the destination address. Therefore, the valid combinations of flits are HeadTail, Head-Tail, and Head-Body(any number)-Tail. Since there are four kinds of flits, a 2-bit identifier is required. In the implementation, the identifiers are as follows: HeadTail - 10, Head - 01, Body - 00, Tail - 11. The All 0's and All 1's vector can be sent as a Body flit and Tail flit respectively, with both their payloads containing zeros. Ideally, this is the minimum number of vectors required to achieve complete coverage of stuck-at faults. However, due to the constraint described above, a Head flit is required to be sent with its payload containing zeros, so that the subsequent flits can be routed to the correct destination. This results in three test packets being sent to other routers, so that any stuck-at fault on the link can be identified.

**Ping Pong process for the four directions**

Consider that the routers follow a pattern East-South-West-North in the ping-pong process. When the system is reset, all routers send these three flits, called as "Ping", one after the other to their east neighbours. The process is completely distributed and all routers send the flits in parallel. This implies that while routers are sending "Ping" flits in their east direction, they are also receiving "Ping" flits from their west direction. Therefore, comparisons are done while the next flit is prepared. The process of updating status registers can be explained by moving through the steps with respect to router 7, which sends three flits to router 8. Also, router 7 receives three flits from router 6 (due to parallel operations). If the outlink between routers 7 and 8 is broken, router 8 will not receive any data and the west inlink status bit in router 8 remains 0. If the flit is received, comparison occurs as follows. In the first cycle, the Head flit is received and since it is just a direction carrier, there is no comparison. In the next cycle, the Body flit is received and is compared with an All 0's flit. If there was a stuck-at-1 fault on the link, this comparison fails and the status bit remains 0. In the next cycle, a Tail flit is received and is compared with an All 1's flit. If there was a stuck-at-0 fault on the

link, this comparison fails. The west status bit in router 7 is updated as 1 and a pong (response) is generated, only if both the comparisons succeed.

It is important to note which bits of the status register are being updated in the routers at this stage. In all the routers, the inlink (with respect to them) in the west direction is updated as 1 if the three flits received are correct. A pong is generated in this case and it updates the outlink (with respect to the routers) in the east direction as 1. This is because a pong is generated only if the destination router received all flits correctly over the concerned link, which is the outlink in that direction for the source router. Therefore, in the first iteration of the process, the bits updated in each router are the west inlink and east outlink bits.

The same procedure occurs in the next iteration when all routers send three flits in their south direction. If both the links are healthy, the bits updated in this iteration are the north inlink and south outlink bits. In the next iteration, all routers send three flits in their west direction. If both the links are healthy, the bits updated in this iteration are the east inlink and west outlink bits. Following this, all routers send three flits in their north direction. If both the links are healthy, the bits updated in this iteration are the south inlink and north outlink bits.

The ping pong process should be performed in all four directions, which ensures that each router has the status of all links connected to it, This in turn offers the advantage of sending just one pong. Receipt of a pong is used to update the outlink status in a particular direction at the router. Although the pong is received over the inlink, the inlink status will be updated in a different iteration (when source and destination routers are reversed). This saves two flits in each iteration, thereby optimizing the process. Further, no comparison is required when a pong is received. This is because a high valid signal when sending the pong is sufficient to infer that the outlink was healthy.

Figure 2.2 shows the updation of the status register for a particular case of faulty inlink in Router 7's south direction. Let us consider both the cases - inlink is broken or inlink has a stuck-at fault. For the first case, the figure highlights an inherent limitation of the ping pong process, as a router falsely marks its outlink in a direction as faulty even if only its inlink in that direction was faulty. This occurs because both the "Ping" and the "Pong" are not received by the router, due to which the router cannot identify a healthy outlink. For the second case, accuracy is improved since the pong is received by

```
/* All routers do the following in parallel */
Initialize status of outlinks and inlinks in all four directions to unhealthy
for Direction d = east, south, west to north  do
   Send Ping to router in direction d
   if Pong received from router in direction d then
      Update Outlink of direction d as healthy
   end if
   if Ping received from router in direction d then
      Send Pong to router in direction d
      Update Inlink of direction d as healthy
   end if
end for
Send updated status of outlinks and inlinks to the control unit
```

**Algorithm 1:** Assessment of Health of Links

the router. Therefore, it can identify a healthy outlink even if the inlink has a stuck-at fault. The pseudo code is presented above.

### Ping Pong process for the ingress/egress ports

It is essential that the health of the links connecting a router and a CR is determined, since applications are launched on the CR. Hence, a ping pong process is required to test these two links. Testing these links is different from the previous approach since there is no parallelism involved. The test patterns and extra hardware required will be apparent once the structure beyond the ingress/egress ports is explained.

The two links connect the router and the CR through an interface called the Network Interface (NI). The NI consists of two separate modules for each direction. The toNoC module connects the output from the CR to the ingress port, after breaking the packet into multiple flits using a "flitizer". The fromNoC module connects the egress port to the CR, after assembling the flits constituting a packet using a "packetizer". Since link testing should not involve the CR (which is not tested yet), the hardware module performing BIST of these links should reside in the NI. This module is responsible for communicating with the main BIST module in the router and determining the status of the links. It requires the contents of the outlink (with respect to the router) as its input and sends appropriate response packets by multiplexing the existing inlink from the toNoC module.

As mentioned in the routing function section, when $\Delta X$ and $\Delta Y$ are 0, the flit is routed to the egress port. This means that every flit which is destined for a particular router

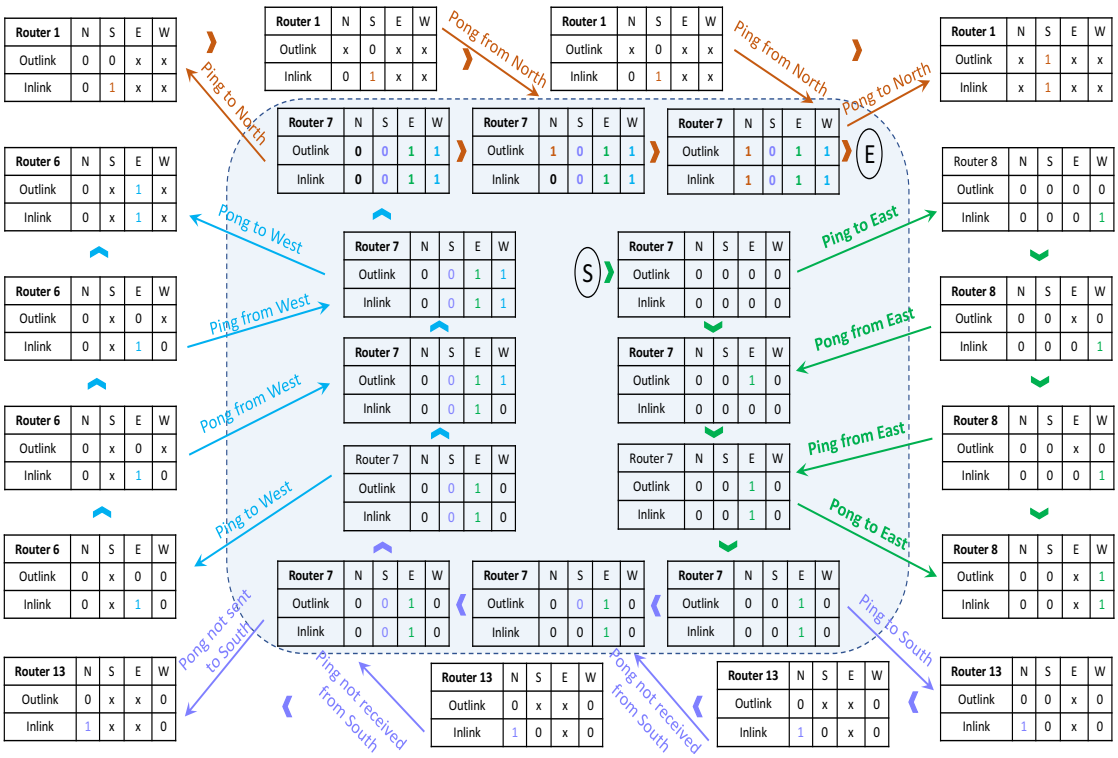



Figure 3.2: Example of a ping-pong process with one faulty link

reaches the NI. This creates a situation during the ping pong process where the BIST module in the NI needs to distinguish between flits intended for it (so that a response can be generated appropriately), and flits that have reached it solely because the router is the destination. This is because each flit sent in any direction in the process will move into the NI in the second cycle, and the state machine in the NI BIST module should not respond to these flits. There are two ways of distinguishing between these flits and the pros and cons of each method are explained below.

### Method 1 - Timing based flit identification

One solution is to have a timeout at each NI BIST module corresponding to the time it takes for the first ping pong process to complete. If the timeout starts decrementing on reset, the module will be ready to accept the correct flits at the end of the timeout.

The advantage of this method is that it has a small hardware overhead (with just a down counter to be added), and there is no potential for faults in links to affect the process.

The disadvantage of this method is that the same reset signal which was given to the main BIST module in the router also needs to be given to this module. Further, since the entire process works only if all the BIST modules start at the same time, one global

reset is required to be given to all routers. Considering the circuit-level view, this signal would have an impractical fanout and would not arrive at all routers at the same time. Therefore, adding more fanouts to the signal would not be practical.

**Method 2 - Event based flit identification**

The second solution is to use one bit in the flit to indicate if it is intended for the NI BIST module. Upon receiving the flit, the BIST module would check if the bit is 1 and correspondingly advance the state machine.

The major disadvantage of this method is that the indicator bit can be a victim of a stuck-at fault, which can lead to false results. For example, if the outlink at the egress port had a stuck-at-1 fault, all flits received by the BIST module would have this bit as 1. This would lead to a situation where the module responds to all flits it receives, thereby leading to potentially false results.

The advantage of this method is that since there is no dependence on a global reset, the circuit-level implementation would be feasible.

**Method choice and implementation**

Since the ultimate goal is to have an SoC implementation of the MP-SoC, a feasible circuit implementation is preferred. The timing based implementation can be used in a situation where the control unit sends reset packets to all routers such that the ping pong process starts at the same time. This would require different timeouts for different routers, but would eliminate the requirement of a global reset signal. This implementation can be looked at in future work. For the current implementation, the event based identification is used.

In order to use the event based flit identification, the NI BIST module needs to identify false positives that occur in case of a stuck-at-1 fault. This is done by using a different payload for the flits intended for the module. As mentioned earlier, the payloads for all ping and pong flits are packed with 0's. By taking advantage of this fact and packing the NI link testing flits with payloads containing all 1's, the false positives can be identified by comparison. Although this is a fairly robust identification, it is not a perfect one. In the case of stuck-at-1 faults in all the payload bits along with the indicator bit, the state machine would advance even if the flit was not intended for the module. For this situation to occur, a large number of bits need to be stuck-at-1. Since the probability of

this situation is very low, it can be reasonably assumed that it does not occur.

Using an example, it can be shown that if the above mentioned situation does not occur, there is no possibility of false results. A flit which is not intended for the NI BIST module would normally have the indicator bit as 0 and payload as all 0's. In the case of a stuck-at-1 fault at the indicator bit, there are two cases - the original bit was 0, or the original bit was 1. In the first case, the module would receive a flit with the indicator bit as 1 and payload as all 0's, and the comparison fails. As a result, the state machine does not advance and no pong is generated. In the second case, the module would receive a flit with the indicator bit as 1 and payload as all 1's, and the comparison succeeds. As a result, the state machine advances and the next two flits are compared to identify stuck-at faults. Here, the comparison fails and no pong is generated.

When the NI BIST module receives ping flits which are intended for it, the state machine is activated and compares the next two flits (All 0's and All 1's) to identify stuck-at faults. If the comparison succeeds, three pong flits are sent to the main BIST module. The reason for generating three pongs is because there is no requirement for source-destination reversal and status registers at this module. Further, since there is no parallelism involved, both inlink and outlink updates happen at the main BIST module and at the same time. Receipt of the first pong denotes that the outlink is healthy. Successive pongs are compared for All 0's and All 1's respectively. If the comparison succeeds, inlink status register is updated as 1.

## 3.2 Consolidation

Once the ping-pong process is completed, each router has 10 bits of information regarding the status of its links. For a distributed system, the optimal method for consolidation would be to have routers send their information to the control unit at the end of specified timeouts. These timeouts are customized and supplied to routers when they are instantiated. The consolidation has to proceed in a sequential order i.e a router should send its information only after the preceding router's information has reached the control unit. This is done for the following reason. The control unit needs to store the status information in memory so that the software part can subsequently use the information to determine availability. Conflicts between information from multiple routers can create

a situation wherein wrong information is stored at a particular memory location. Sequential consolidation avoids such situations and hence is preferred even if the number of cycles is larger.

The consolidation step requires error checking in the case of stuck-at faults. Consider a situation where a faulty links implies a broken link, and stuck-at faults are not considered. If there are no broken links in the path from a router to the control unit, its information is guaranteed to reach the control unit. Conversely, if there is atleast one broken link on the path to control unit, the information from a router will not reach the control unit. As will be explained in the next chapter, loss of certain routers' information does not impact the determination of availability in the sense that no unavailable routers are marked as available. However, depending on the pattern of faults, marking available routers as unavailable is a possibility. This will be explained using examples in the next chapter.

Introducing stuck-at faults has a big impact on the interpretation of information at the control unit. Since information flows over potentially faulty links to reach the control unit, it is imperative to check for its correctness. Figure 3.3 shows a portion of the execution fabric. Let router 17 send its information to the control unit over links shown in green. There are two cases of stuck-at faults which can corrupt this information. These are explained as follows.

**Stuck-at-1 fault in any link at a bit containing status information**

In the case of a stuck-at-1 fault, the control unit is guaranteed to read the corresponding status bit as 1. However, the control unit cannot identify if the original bit sent by the router was 1 or 0. Also, as mentioned earlier, a high status bit denotes that the link is not faulty, and for determining availability, it should be ensured that there are no false positives (unavailable router being marked as available). Therefore, ambiguity dictates that the value of the corresponding status bit should be stored as 0.

The next requirement is to identify the bit that is a victim of a stuck-at-1 fault. The routers need to send an extra flit immediately following the flit containing status information. The extra flit should contain 0's in all the bits corresponding to status bits in the other flit. By the time this flit reaches the control unit, the bit(s) that was(were) victim(s) of stuck-at-1 fault(s) would have changed from 0 to 1. Since the control unit

would be expecting all 0's, a comparison would identify all faulty bits and corresponding status bits can be made 0 to eliminate false positives.

#### Stuck-at-0 fault in any link at a bit containing status information

In the case of a stuck-at-0 fault, the control unit is guaranteed to read the corresponding status bit as 0. However, the control unit cannot identify if the original bit sent by the router was 0 or 1.

In contrast to the previous case, no further check is required to identify the bits that are victims of stuck-at-0 faults. This is because the corresponding status bits would already be forced to 0 (implying faulty link). Therefore, there is no possibility of false positives.

#### Comparison at control unit

The control unit receives two flits from each router. The first flit contains status information of the router's links and the second flit contains 0's in the corresponding bits. By ANDing the information bits from the first flit with the complement of the corresponding bits in the second flit, all false positives are eliminated and the resulting 10 bits are stored in the corresponding memory location.

## 3.3 Decision Making

Using the BIST information, usable regions can be identified to launch applications. This decision can be taken by the software component of the control unit, thereby eliminating the requirement of extra hardware. There are three stages in decision making - Reachability analysis, faulty link analysis and determination of available regions.

#### Reachability Analysis

Reachability of a router is determined by the health of two paths - from control unit to router and vice versa. The rationale behind this is that, when an application is launched on multiple routers, one router is tasked with sending the end of application

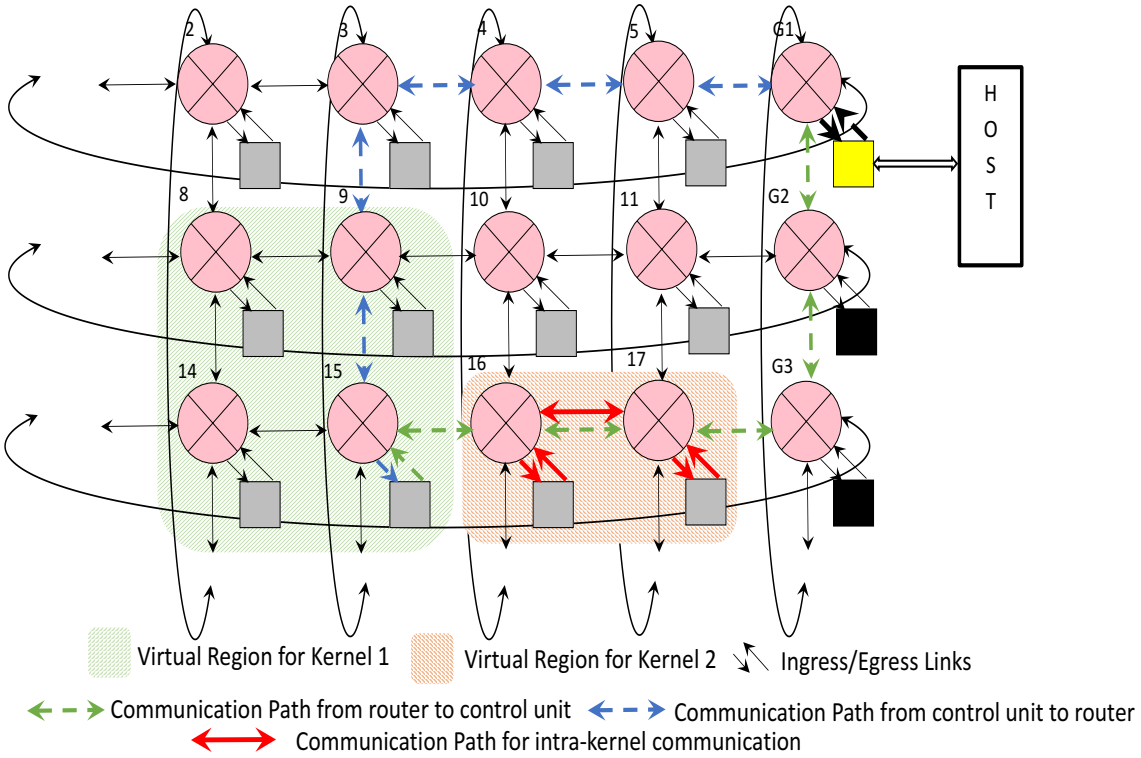



Figure 3.3: Example of a pass-through router

signal back to the control unit. Since any router can perform this task depending on the region allocated, all routers in the chosen sub-fabric need to have fault-free paths for communication between themselves and the control unit. As explained above, there exist two different paths for communication from control unit to router and from router to control unit. As part of the reachability analysis, it is important to check the status of both the paths, and mark a router as available only if both the paths are fault-free.

There is an important distinction to be made here. Even if a router is marked as unavaialable, it can still act as a "pass-through" router for other routers if the required links are fault-free. Let us consider an example as shown in Figure 3.3. A small region of the fabric explained earlier is shown. By following X-Y routing, router 16 would require its north inlink to be healthy in order to be able to receive packets from the control unit. Let us assume that this was the only faulty link in the entire NoC. By reachability analysis, router 16 would be marked as unavailable. However, it can still act as a pass-through router for router 15, which uses router 16's healthy east outlink to communicate to the control unit.

The reachability is determined by the information obtained during the ping-pong process. Since the status of one link is given by two routers, an OR operation of the two

gives the accurate status. Depending on the pattern of faults, there is a reasonable chance that information from atleast one of the two routers holding the status of a link would reach the control unit. Further, since there are no false positives (status of faulty link being stored as 1), an OR operation would suffice. The implementation of this algorithm is as follows. Two bitmaps (string of bits with the $i^{th}$ bit denoting status of the $i^{th}$ router, 0 being unavailable and 1 being available) are initialized with value 0 (to rule out false positives), one for the receive reachability (routers being able to send packets to control unit) and the other for transmit reachability (control unit being able to send packets to routers).

As a consequence of the toroidal connection, the execution fabric is virtually divided into four regions, since the shortest path for the bottom or the left half of the fabric will involve the corresponding toroidal connection. Therefore, for communication from routers to control unit, the top half of the fabric uses the south inlinks of corresponding gateway routers and the bottom half of the fabric uses the north inlinks of corresponding gateway routers. Similarly, for communication from control unit to routers, the left half of the fabric uses the west inlinks of corresponding routers in the first row, and the right half of the fabric uses the east inlinks of corresponding routers in the first row.

For the receive bitmap, the vertical inlinks of the gateway routers are checked first. If any vertical inlink is faulty, routers in all the rows that are dependent on this link are unavailable since they will either not be able to send a packet to the control unit or send an incorrect packet (due to stuck-at faults). This is a consequence of X-Y routing (minimizing X and then minimizing Y) and the reason why the health of vertical inlinks of the gateway routers are crucial to the health of the NoC. If a vertical inlink is healthy, the corresponding row is tested by checking the horizontal inlinks. Routers are marked unavailable based on faulty horizontal inlinks.

For the transmit bitmap, the horizontal inlinks of the routers in the first row are checked. If any horizontal inlink is faulty, routers in that column are unavailable since they will either not be able to receive a packet from the control unit or receive an incorrect packet (due to stuck-at faults). This is a consequence of X-Y routing (minimizing X and then minimizing Y). If a horizontal inlink is healthy, the corresponding column is tested by checking the vertical inlinks. Routers are marked unavailable based on faulty vertical inlinks.

The final outcome of this stage is a bitmap that represents reachability of each router, calculated by bitwise AND of the receive and transmit bitmaps.

### Faulty link analysis

This stage is used to recognize faults in links that are not used in any receive or transmit between routers and the control unit. The importance of these links arise at the time of identifying a potential sub-fabric for launching applications. As discussed earlier, an application can be launched in a rectangular region of the fabric. An important observation is that all intra-application communication happens inside this "virtual region" by utilizing the links connecting the routers in the rectangular region. This implies that health of all these links need to be checked before an application can be launched in that region.

Implementation-wise, two more bitmaps are generated, one for the horizontal links and another for the vertical links. At this stage, it is unnecessary to make a distinction between inlink and outlink in any direction, since a fault in either one of them would render the region unavailable. From the link information, the horizontal bitmap is updated with the combined status of the east links of a router, and the vertical bitmap is updated with the combined status of the south links of a router.

### Determination of available regions

Once a potential region is identified by the control unit, the reachability of all the routers is checked using the corresponding bitmap, followed by checking of the health of the enclosed links. Until the above two requirements are satisfied, the control unit searches for other potential regions. Once the requirements are satisfied, the control unit allocates the region for the application.

# CHAPTER 4

# Results and Examples

In this chapter, details of the procedure such as hardware overheads and latency of individual processes are presented. A few examples are also presented to demonstrate the working of availability analysis.

## 4.1 Execution time for ping pong process

Figure 4.1 shows the execution time in terms of number of cycles for the ping pong process with respect to fabric scaling. Since the process is fully distributed, i.e all routers perform the same operation at exactly the same time, the number of cycles taken is fixed regardless of how big the fabric is. This is a major advantage of the algorithm because status of all links are known 24 cycles after reset is applied. While this process is highly efficient in terms of execution time, having information in a distributed manner affects the execution time of the consolidation process as explained below.

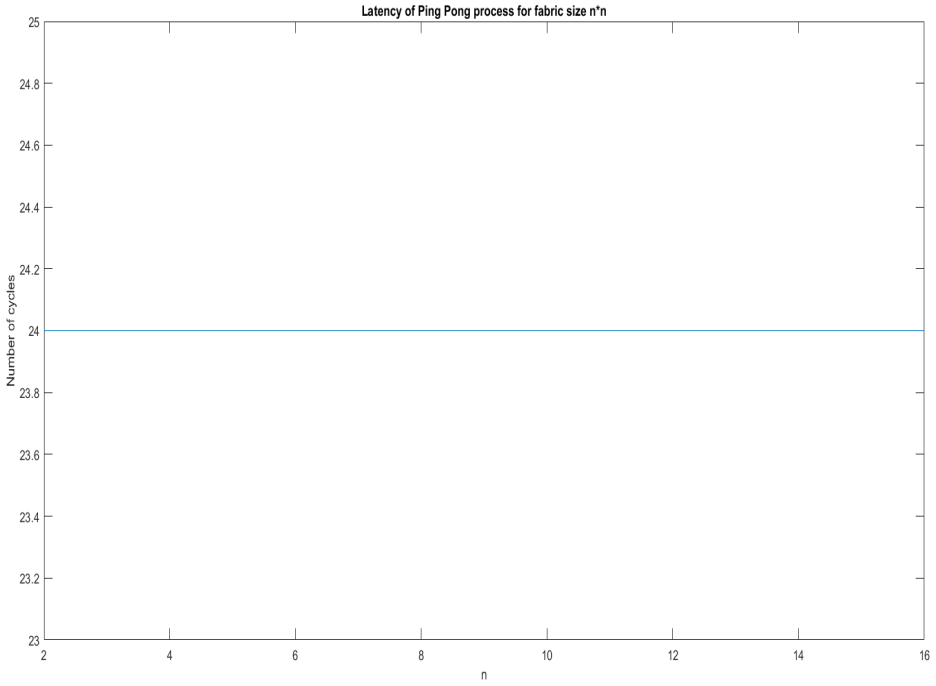



Figure 4.1: Number of cycles required for ping pong process with fabric scaling

## 4.2 Execution time for consolidation process

In this process each router sends its information to the control unit at the end of a specified timeout. This timeout is customized for each router during its instantiation. To avoid any overlap between information from two routers, a sequential order is followed i.e, a router sends its information only when the previous router's information has reached the control unit. This is a conservative approach which can be optimized by allowing the routers' information to move into the control unit one cycle after another. The order of sending information starts with router 0, moves through columns of the first row, then moves through columns of the second row (from left to right), and eventually ends at router G6.

Figure 4.2 shows the scaling of the number of cycles required for consolidation process with respect to the fabric size. The graph follows a quadratic (n*n) increase in number of cycles as n increases. This is because each router needs to send its information to the control unit using X-Y routing, which incurs a lot of latency. As mentioned above, this number can be slightly reduced by optimizing the consolidation process.

The time taken by routers to send their information to the control unit for a fabric of size n*n is calculated as follows:

**n - Even**

No. of cycles taken by a flit to travel from each router to router G1:

First row - 1 + 2 + ... + n/2 + n/2 + (n-2)/2 + ... + 1 + 0 (0 is for router G1)

Second row - 2 + 3 + ... + (n+2)/2 + (n+2)/2 + ... + 2 + 1

.

.

Middle row - (n+2)/2 + (n+4)/2 + ... (depends on n) + ... + (n+4)/2 + (n+2)/2 + n/2

.

.

Last row - 2 + 3 + ... + (n+2)/2 + (n+2)/2 + ... + 2 + 1

**n - Odd**

No. of cycles taken by a flit to travel from each router to router G1:

First row - 1 + 2 + ... + n/2 + (n+2)/2 + n/2 + ... + 1 + 0 (0 is for router G1)

Second row - 2 + 3 + ... + (n+2)/2 + (n+4)/2 + (n+2)/2 + ... + 2 + 1

.

.

Middle row - (n+2)/2 + (n+4)/2 + ... (depends on n) + ... + (n+4)/2 + (n+2)/2 + n/2

.

.

Last row - 2 + 3 + ... + (n+2)/2 + (n+4)/2 + (n+2)/2 + ... + 2 + 1

Apart from this, flits from all routers take 2 additional cycles to reach control unit. Therefore, the sum of the above partial sums along with n*(n+1)*2 (to reach control unit) gives the total number of cycles used in the consolidation phase.

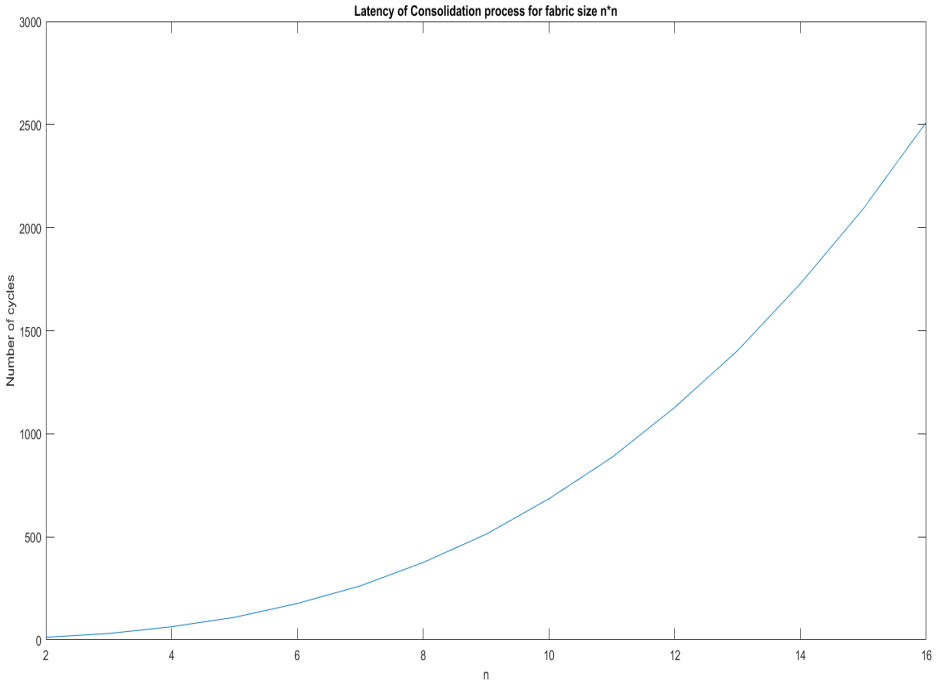



Figure 4.2: Number of cycles required for consolidation process with fabric scaling

The software analysis identifies available routers by moving through the same paths used for communication between routers and control unit. Since the fabric is divided into four quadrants as shown in Figure 4.3, the algorithm has a complexity of O(n/2*n/2).

## 4.3 Implementation and hardware overheads

A 3*3 execution fabric along with one column of gateway routers was emulated on a Xilinx Virtex-7 2000T FPGA. The below table presents the hardware overheads incurred by the BIST modules.

| Component | Without BIST | With BIST | Percentage increase |
|---|---|---|---|
| LUT | 693685 | 708955 | 2.2% |
| LUTRAM | 55147 | 55491 | 0.624% |
| FF | 374687 | 377952 | 0.871% |
| BRAM | 456.5 | 672.5 | 47.32% |
| DSP | 508 | 508 | 0% |
| BUFG | 12 | 12 | 0% |
| MMCM | 2 | 2 | 0% |
| PLL | 2 | 2 | 0% |

As can be seen from the table, there is an insignificant increase in the number of components with the addition of the BIST module. However, BRAM has incurred a 47.3% increase. This can be due to two reasons. One, the number of BRAMS used is quite small. Therefore, even a small increase in number leads to a large percentage increase. Two, in the ping pong and consolidation processes, a large number of vectors and registers are used for storage of link status information. This could have resulted in an increase in the number of BRAMs.

**Examples showing the working of availability analysis**

Figure 4.3 shows the execution fabric with the virtual quadrants created due to X-Y routing. For an even number of rows or columns, there are two potential paths for communication between a router and control unit. One such path is assumed in the figure. All examples are explained using this figure. All routers in quadrant I use their east outlinks and north outlinks of corresponding gateway routers (if any) to communicate with the control unit. All routers in quadrant II use their west outlinks (toroidal connection at the first column routers) and north outlinks of corresponding gateway routers (if any) to communicate with the control unit. Similarly quadrant III routers use their west outlinks (toroidal connection at the first column routers) and south outlinks (toroidal connection

at gateway router of last row), whereas quadrant IV routers use their east outlinks and south outlinks (toroidal connection at gateway router of last row) for communication with the control unit.

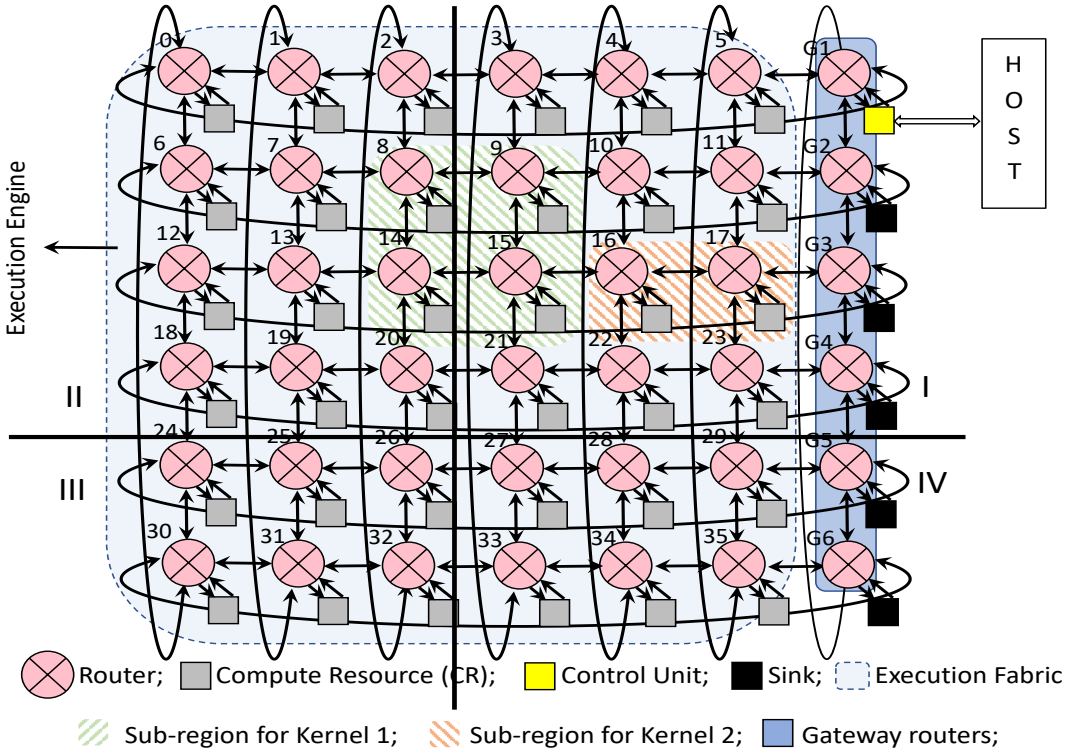



Figure 4.3: Virtual divisions of the fabric due to X-Y routing

## 4.4 Example 1 - Faulty vertical inlink in gateway column

The vertical links in the gateway column are most crucial since they are used for transmitting information from many routers to the control unit. In the following cases, both broken link fault and stuck-at fault are considered individually to demonstrate the difference between the analysis for the two types of faults. It is also assumed that the link under question is the only fault in the NoC.

### 4.4.1 Case 1 - Broken south inlink of router G3

During the ping pong process, the link is identified as faulty by routers G3 and G4. During consolidation, router G3's information reaches the control unit, whereas router

G4's information does not (due to broken link). Since the status of one link is present at two routers, there is a reasonable chance that atleast one of the routers would send the information to the control unit. If both the routers are unable to send the information, it leads to two cases, one in which the final decision is not impacted by the loss of this information, and another in which the decision is impacted. These cases will be discussed in detail in subsequent sections.

In the present case, the software part of the control unit sees that the south inlink of G3 is faulty. As a result, during receive reachability analysis, all routers dependent on this link to transfer their information to control unit are marked as unavailable, namely routers 18 to 23. Transmit reachability, on the other hand, marks all routers as available. Therefore, in the final reachability bitmap, all fourth row (from top) routers are marked as unavailable. Faulty link analysis yields only one bit in the vertical bitmap as 0. Since this link is not part of the execution fabric, it does not impact launching of applications in available parts of the execution fabric.

Result of the availability analysis is shown in figure 4.4. Both transmit and receive reachability analyses are shown along with the aggregated result obtained by ANDing the two bitmaps. The result denoted as final at the bottom of the figure combines both reachability and faulty link analysis in one matrix. Let elements of the matrix be denoted by (x,y), x and y being the normal coordinates. Every (odd, odd) element denotes the availability of a router. Every (odd, even) element denotes the status of corresponding vertical links. Every (even, odd) element denotes the status of corresponding horizontal links. Finally, every (even, even) element denotes an empty space in the topology and does not feature in analysis. This explanation is valid for all such figures.

### 4.4.2 Case 2 - Stuck-at fault in south inlink of router G3

In this case, information from routers G3 and G4 reach the control unit. However, care must be taken while interpreting the results. The information sent by G3 is accurate and the control unit can determine that the row of routers containing G3 is available. However, information of router G4 is corrupted by a stuck-at fault at one of the bits. There is a mechanism in place as discussed earlier that eliminates false positives when bits are corrupted. After this is done, the information is stored at the corresponding memory

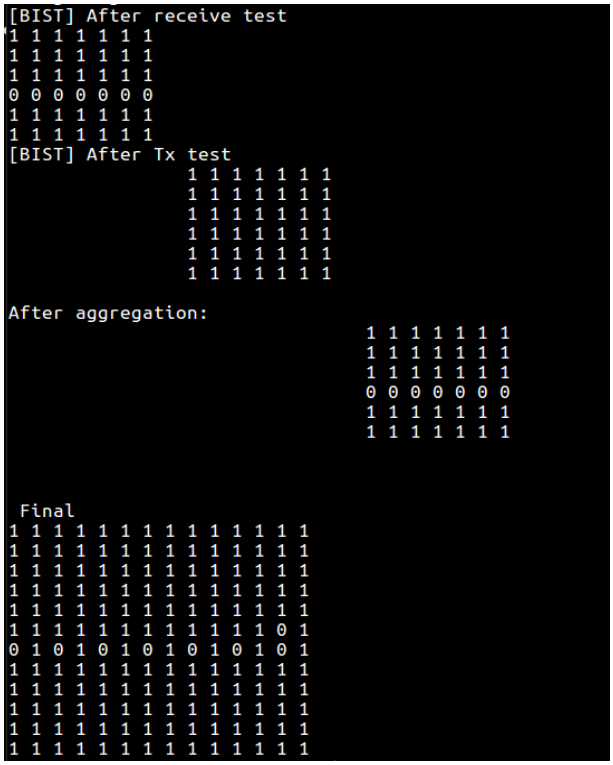



Figure 4.4: Availability analysis of Example 1, Case 1

location. The decision making stage marks the fourth row routers as unavailable.

There are three cases in which the stuck-at fault can potentially impact decision making: stuck-at fault at the bit denoting status of the link under question, stuck-at fault at a bit carrying status information other than that of the link under question, and stuck-at-1 fault at a bit not carrying status information. It is obvious that the last case does not require any further action, because no information is corrupted. The first case also does not require much scrutiny since the decision is made using inlink status information from the other router.

The second case is interesting because information about another link is corrupted and hence stored as 0 (faulty). However, this is a limitation imposed due to the use of potentially faulty links to transfer information to the control unit, and cannot be circumvented. Depending on which bit was corrupted, other reachability analyses could be affected. For example, let us consider that the bit containing information about router 23's north inlink is corrupted. Therefore, in transmit reachability, router 23 is marked as unavailable. In this case, it does not impact accuracy of determining available routers. However, it can be shown using other examples that some routers can be incorrectly marked as unavailable due to such stuck-at faults.

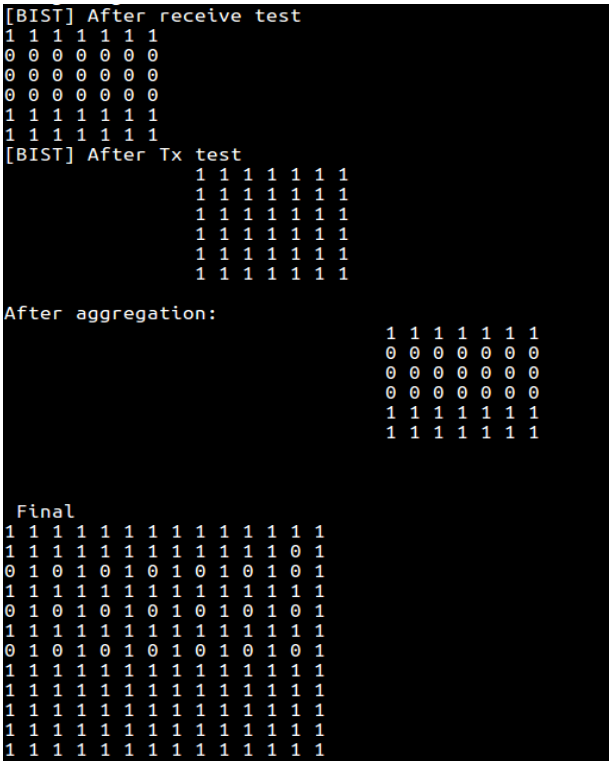



Figure 4.5: Availability analysis of Example 1, Case 3

### 4.4.3 Case 3 - Broken south inlink of router G1

This is the most crucial link for availability of the execution fabric. Excluding the first row of routers, all routers in the I and II quadrants are dependent on this link to send their information to the control unit. If this link is broken, the control unit itself has this information at the end of the ping pong process. Subsequently, receive reachability analysis marks the above mentioned routers as unavailable. Lack of information from these routers does not impact the decision since none of them are used as pass-through routers for any of the III and IV quadrant routers. Figure 4.5 shows the result of availability analysis for this case.

### 4.4.4 Case 4 - Stuck-at fault in south inlink of G1

This case differs from the previous one in that information from all the concerned routers reaches the control unit. Due to the stuck-at fault (either 1 or 0), some links would be incorrectly marked as faulty. However, since the I and II quadrants excluding the first row are already marked unavailable by the receive reachability analysis, and none of them are pass-through routers for III and IV quadrant routers, incorrect information does not

impact decision making.

## 4.5 Example 2 - Faulty horizontal inlink in first row

The horizontal links in the first row are also crucial since they are used for transmitting information from the control unit to many routers. In the following cases, both broken link fault and stuck-at fault are considered individually to demonstrate the difference between the analysis for the two types of faults. It is also assumed that the link under question is the only fault in the NoC.

### 4.5.1 Case 1 - Broken east inlink of router 3

During the ping pong process, this link is identified as faulty by routers 3 and 4. Both the routers are able to send their information to the control unit because the faulty link is not on their path to control unit. At the control unit, the transmit reachability analysis identifies that routers in the column containing router 3 are unavailable since they cannot be reached by the control unit. There is no loss of information in this case. Figure 4.6 shows the result of availability analysis.

### 4.5.2 Case 2 - Stuck-at fault in east inlink of router 3

During the ping pong process, this link is identified as faulty by routers 3 and 4. The control unit receives correct information from both the routers since there is no stuck-at fault in their path to control unit. During availability analysis, transmit reachability identifies that routers in the column containing router 3 are unavailable since they cannot be reached by the control unit. There is no possibility of corrupted information here since the stuck-at fault is on a link which is not used by any router to send its information to control unit.

### 4.5.3 Case 3 - Broken east inlink of router 5

This link is quite crucial since it is used by the control unit to communicate with half of the execution fabric (I and IV quadrants). During the ping pong process, the link is

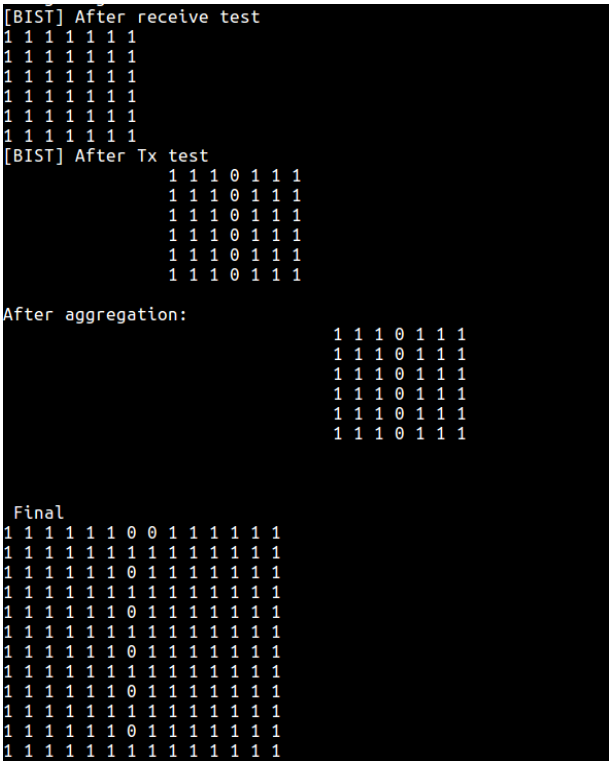



Figure 4.6: Availability analysis of Example 2, Case 1

identified as faulty by routers 5 and G1. Since router 5's information is accessible by the control unit, the transmit reachability analysis identifies the I and IV quadrant routers as unavailable. Figure 4.7 shows the result of availability analysis for this case.

### 4.5.4 Case 4 - Stuck-at fault in east inlink of router 5

This is similar to the previous case in that half of the execution fabric is marked as unavailable. Further, corruption of information by stuck-at fault does not impact the decision since it would only affect routers that are already marked as unavailable.

It is apparent from the above examples that a critical fault is one on the path from a router to the control unit, and the criticality increases if the fault is closer to the control unit and on the gateway column. Faults on the transmit path to a router also have a major impact on the availability of the fabric if they are close to the control unit. Further, there was no impact of stuck-at faults on the decision in the above examples, and only one fault was considered. In the following examples, corner cases will be explained and the impact of stuck-at faults will also be discussed.

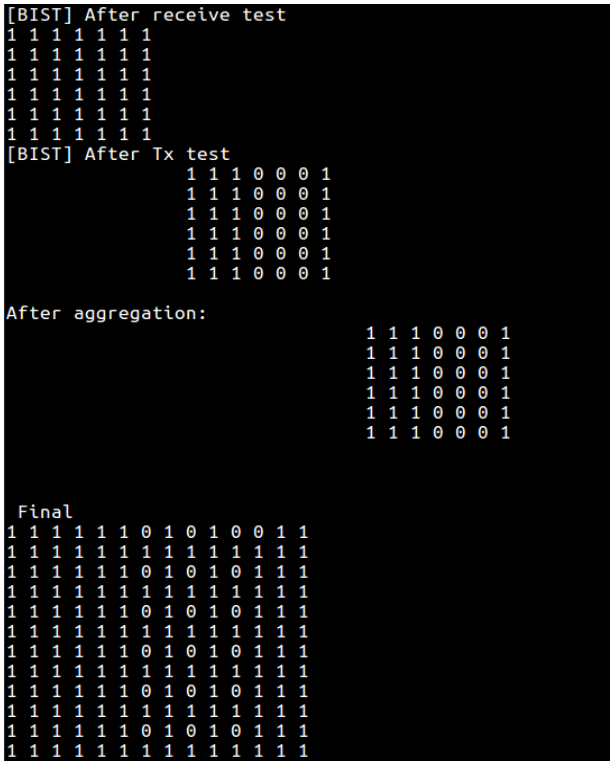



Figure 4.7: Availability analysis of Example 2, Case 3

## 4.6 Example 3 - Faulty west inlink of router G1

In this example, a corner case is discussed which has a major impact on the availability of the system. It is assumed that the faulty link mentioned is the only fault in the NoC.

### 4.6.1 Case 1 - Broken west inlink of router G1

In this case, the fault is very close to the control unit. Due to the nature of the ping pong process and as explained in the corresponding section, router G1 identifies both the inlink and outlink in its west direction as faulty. Router 5 on the other hand identifies its east inlink (west outlink for router G1) as healthy. However, router 5's information does not reach the control unit since the corresponding link is broken. Similarly, information from all the first row routers in the I quadrant fails to reach the control unit.

In this situation, all the horizontal inlinks in the first row of the first quadrant are marked as faulty, because the corresponding memory locations were initialized as 0 and were not updated due to loss of information. As a result, transmit reachability analysis identifies all the routers in the I and IV quadrants (excluding gateway column) as unavailable. This is a highly cnservative estimation caused by loss of information. Ideally,

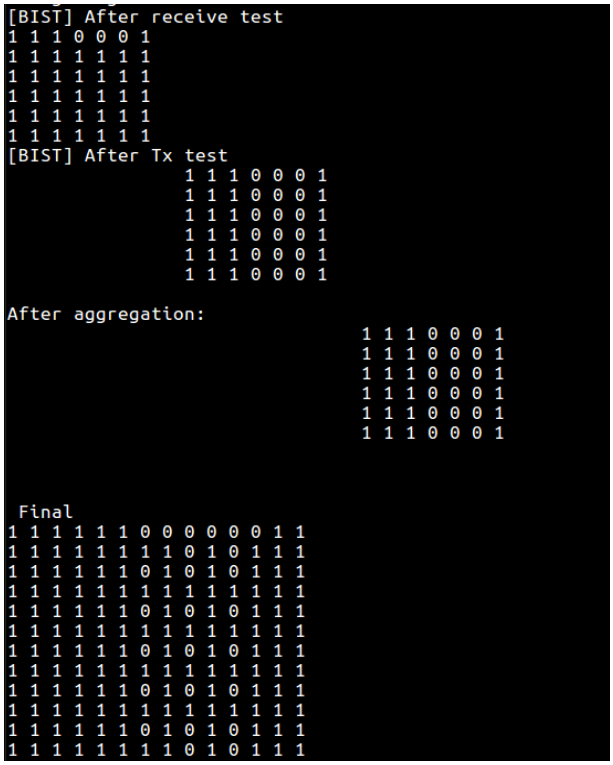



Figure 4.8: Availability analysis of Example 3, Case 1

the decision should have conveyed that routers 3, 4 and 5 were unavailable. However, the actual decision made is that half of the execution fabric is unavailable. Figure 4.8 shows the result of availability analysis for this case.

### 4.6.2 Case 2 - Stuck-at fault in west inlink of router G1

Having a stuck-at on the link instead of a broken link improves the accuracy slightly, however there are other repercussions. Router G1 now identifies its west outlink as healthy. Therefore, the column of routers containing router 5 is guaranteed to be marked as available. The state of the other columns is dependent on the location of the stuck-at fault. If the stuck-at fault is located at a bit denoting the status of the routers' west inlink, north inlink or south inlink, only the first row routers are affected, and the other routers in all columns are marked as available. If the stuck-at fault is located at a bit denoting the status of the routers' east inlink, all the column routers are marked as unavailable. If the stuck-at fault is located at a bit denoting the status of the routers' north outlink or south outlink, it may not affect the other routers in the columns. This is because if the control unit receives uncorrupted information from routers in the last row, it is guaranteed to mark all routers (excluding first row routers) in the I and IV

quadrants as available.

Therefore, it is apparent that in certain cases, having stuck-at faults instead of broken link faults leads to better accuracy of availability analysis.

## 4.7 Example 4 - Faulty east outlink of router 11

The importance of collecting information of one link at both the routers connected by it is shown in this example.

### 4.7.1 Case 1 - Broken link

In this case, information from routers 9 to 11 does not reach the control unit. As a result, the status of north inlinks of these routers is not updated at the corresponding memory locations. However, routers 3 to 5 convey status of their south outlinks (same as north inlinks of routers 9 to 11) to the control unit. By identifying that the south outlinks are fault-free, the control unit can correctly mark all routers other than 9, 10 and 11 as available. This shows that a combination of status information of one link (given by two routers) is required to take a correct decision. Figure 4.9 shows the result of availability analysis for this case.

### 4.7.2 Case 2 - Stuck-at fault

In this case, availability analysis is not affected since other routers convey accurate information about the corresponding links.

## 4.8 Example 5 - Faulty north outlink of router 15

In this example, it is shown that even in the presence of a faulty link, the entire execution fabric is marked available.

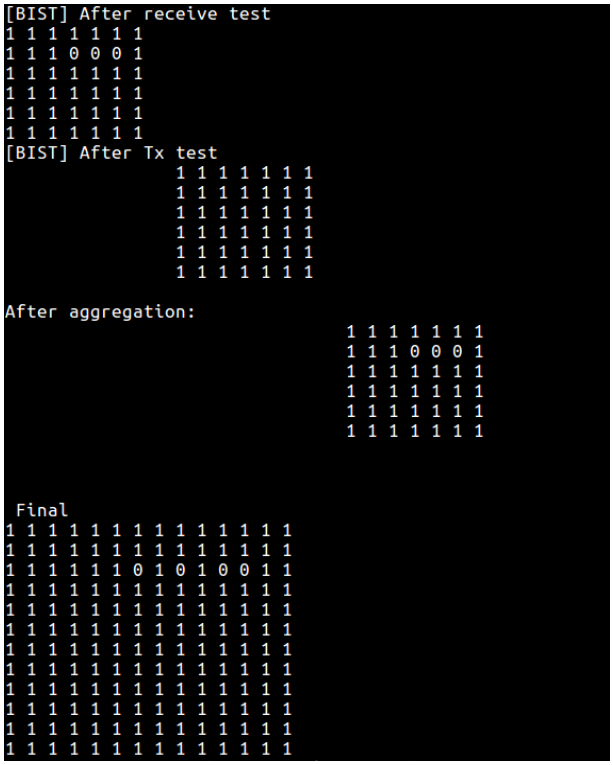



Figure 4.9: Availability analysis of Example 4, Case 1

### 4.8.1 Case 1 - Broken link

This link mentioned is not used by any router to communicate information to the control unit. Therefore, there is no lack of information at the control unit. Further, since the link is not exercised in transmit or receive reachability analysis, all routers are marked as available. In faulty link analysis, the connection between routers 9 and 15 are marked as faulty. Therefore, no applications will be launched in an area containing routers 9 and 15.

### 4.8.2 Case 2 - Stuck-at fault

The above analysis follows for this case as well because no router uses this link to communicate with the control unit. Hence, there is no scope for corruption of information.

All the examples discussed have assumed one faulty link in the NoC, at both critical and less critical locations. The next example discusses a situation of two faulty links in close proximity.

## 4.9 Example 6 - Faulty east outlinks of routers 11 and 17

In this example, it is assumed that there are two faults in the NoC as mentioned. Both cases of broken link and stuck-at fault are discussed.

### 4.9.1 Case 1 - Broken link

In this case of coupled faults (faults on consecutive horizontal links), information from routers 9, 10, 11, 15, 16 and 17 would not reach the control unit and these routers are marked as unavailable. As a result, status of north inlinks of routers 15, 16 and 17 is not updated at the control unit by any of the routers that contain this information. This affects the transmit reachability analysis because routers 21 to 23 are now incorrectly marked as unavailable due to loss of information. The situation is more dire when the coupled faults are closer to the control unit and the fabric size is bigger than 6*6, because a large number of routers are incorrectly marked as unavailable.

### 4.9.2 Case 2 - Stuck-at fault

In the case of stuck-at coupled faults, the impact on decision making depends on the location of the stuck-at faults among the bits containing status information. The above situation arises only if there is a stuck-at fault at the same bit location on both the links. Otherwise, one of the routers' information would be accurate and transmit reachability analysis will not be affected. Consider that there is a stuck-at fault at the same bit location on both links. If the stuck-at fault corrupts the status of any link other than the north inlink and south outlink of routers, it does not affect analysis and hence no routers are incorrectly marked as unavailable. However, if the status bits of north inlink or south outlink are affected, it results in the above situation where a large number of routers are marked unavailable.

## 4.10 Example 7 - Faulty east and west inlinks of router G1

This case is similar to broken link case of Example 3. The main difference is that there is a similar fault on the opposite side of the router. As it was explained in Example 3, a broken west inlink of router G1 would lead to a decision marking half of the execution fabric as unavailable. In this case, the other half of the fabric is also marked as unavailable due to the existence of a mirror image fault. Therefore, even with just two faulty links in the NoC, the entire execution fabric is marked as unavailable. This is an inherent limitation of the ping pong process because of distributed nature of the system.

For a stuck-at fault, the analysis again follows that of Example 3, Case 2. However, both halves of the fabric are equally affected only if the stuck-at faults are in the same bit location on both links.

Therefore, it can be seen that presence of stuck-at faults instead of broken links improves the accuracy of availability analysis. However, there are corner cases which cannot be overcome due to the inherent distributed nature of the system and the fact that status information travels to the control unit over potentially faulty links.

# CHAPTER 5
# Conclusions

## 5.1 Analysis

From the previous chapter, it can be seen that duplicity of status information ensures that in most cases, the correct decision is taken. For some patterns of faults, more number of routers than required are marked as unavailable. However, the important thing is that no router is incorrectly marked as available. This guarantees completion of applications launched on the available portions of the execution fabric.

## 5.2 Future Work

The main drawback of the procedure described is that status information is communicated to the central control unit via potentially faulty links. One way to overcome this drawback would be to have direct connections from each router to the control unit. However, this introduces new complications. Firstly, these connections have to be tested and identified as fault-free. Secondly, scalability is not an option in this scenario since the number of connections increases drastically as the fabric size increases.

Another way of overcoming the drawback is to have a BIST-mode routing. This implies that during BIST process, an alternate path may be used to communicate the status information to the control unit if the original path is faulty. This can prove beneficial in overcoming the coupled faults problem explained in the previous chapter. For example, let us assume BIST-mode routing is defined as follows:
if (link under question != faulty) : Minimize $\Delta$X first, then minimize $\Delta$Y
else : Minimize $\Delta$Y first, then minimize $\Delta$X

Applying this condition to Example 6, it can be seen that information from all routers reaches the control unit. This is because information from routers in the second and third row (from top) reaches the control unit by minimizing $\Delta$Y once the faulty link is encountered. As a result no router is incorrectly marked as unavailable.

While BIST-mode routing in certain scenarios, there are other examples where it is ineffective. Consider Example 3 described earlier. Here, BIST-mode routing is equivalent to normal routing since $\Delta$Y is zero.

In addition to ineffectiveness against certain corner cases, BIST-mode routing incurs changes in the routing algorithm and therefore has hardware overhead. Future work can be aimed at identifying if the hardware overhead is justified based on significant improvement in accuracy of availability analysis.

Moving forward, other components of the system can be tested using availability analysis. For example, the control unit can probe the CRs connected to available routers and test the working of the processor, memory and other functional units. By performing a recursive test, i.e using tested components to test further components, unnecessary testing can be avoided, thereby shortening the testing process and improving energy efficiency.

# REFERENCES

(1) C. Grecu, P. Pande, A. Ivanov, and R. Saleh, BIST for Network-on-Chip Interconnect Infrastructures, in VLSI Test Symposium, 2006. Proceedings.24th IEEE, pp. 6-pp,IEEE, 2006.

(2) C. Grecu, A. Ivanov, R. Saleh, and P. P. Pande, Testing Network-on-Chip Communication Fabrics, IEEE Transactions on Computer Aided Design of Integrated Circuits and Systems, vol. 26, no. 12, p. 2201, 2007.

(3) J. Raik, R. Ubar, and V. Govind, Test Configurations for Diagnosing Faulty Links in NoC Switches, in Test Symposium, 2007. ETS07. 12th IEEE European, pp. 29-34, IEEE, 2007.

(4) M. Hosseinabady, A. Banaiyan, M. N. Bojnordi, and Z. Navabi, A Concurrent Testing Method for NoC Switches, in Proceedings of the conference on Design, automation and test in Europe: Proceedings, pp. 1171-1176, Euro- pean Design and Automation Association, 2006

(5) A. M. Amory, E. Briao,E. Cota, M. Lubaszewski, and F. G. Moraes, A Scalable Test Strategy for Network-on-Chip Routers, in Test Conference, 2005. Proceedings. ITC 2005. IEEE International, pp. 9-pp, IEEE, 2005.

(6) R. Nourmandi-Pour, A. Khadem-Zadeh, and A. M. Rahmani, An IEEE 1149.1-based BIST method for at-speed testing of inter-switch links in network on chip, Microelectronics Journal, vol. 41, no. 7, pp. 417-429, 2010.

(7) J. D. Lee and R. N. Mahapatra, In-field NoC-based SoC testing with Distributed Test Vector Storage, in Computer Design, 2008. ICCD 2008. IEEE International Conference on, pp. 206- 211, IEEE, 2008.

(8) K. Petersen and J. Oberg, Toward a Scalable Test Methodology for 2D-mesh Network-on-Chips, in Proceedings of the conference on Design, Automation and Test in Europe, pp. 367-372, EDA Consortium, 2007

(9) M.Alle, K.Varadarajan, A.Fell, N.Joseph, S.Das, P.Biswas, J.Chetia, A.Rao, S.Nandy, R.Narayan,et al., REDEFINE: Runtime reconfigurable polymorphic ASIC, ACM Transactions on Embedded Computing Systems(TECS), vol. 9, no. 2, p. 11,

2009

(10) M.Alle, K.Varadarajan, A.Fell, S.K. Nandy, and R. Narayan, Compiling Techniques for Coarse Grained Runtime Reconfigurable Architectures, proceedings of the 5th International Workshop on Applied Reconfigurable Computing, Karlsruhe, Germany, March 2009.

(11) A.Fell, P.Biswas, J.Chetia, S.K.Nandy, and R.Narayan, Generic routing rules and a scalable access enhancement for the Network-on-Chip RECONNECT, (proceedings of the SOCC 2009), SOC Conference, September 2009

(12) https://dl.acm.org/citation.cfm?doid=2228360.2228584

(13) William Dally, Brian Towles, Principles and Practices of Interconnection Networks, 2004

(14) Pengju Ren, Michel A. Kinsy, Mengjiao Zhu, Shreeya Khadka, Mihailo Isakov, Aniruddh Ramrakhyani, Tushar Krishna, Nanning Zheng, "FASHION: Fault-Aware Self-Healing Intelligent On-chip Network", 2017